FEEDME × SUC HACKATHON 2026

AI Coding Workshop

CLAUDE CODE NEXT.JS GitHub Vercel



FeedMe
Smart POS System

THE DELIVERY LOOP

The four-step delivery process



01

Describe

One user, one problem, one action.

02

Build

Claude creates a Next.js happy path.

03

Publish

GitHub keeps the team's source code.

04

Deploy

Vercel turns each push into a live build.

Web only Responsive Next.js websites are allowed. Native mobile frameworks are not.

PREFLIGHT

Required setup before the workshop

Finish installation and sign-in at least 48 hours before the session.



Claude Desktop

Claude Code opens



GitHub

`gh auth status`



Vercel

CLI + MCP connected



Starter

`npm run verify`



Team access

Accounts confirmed

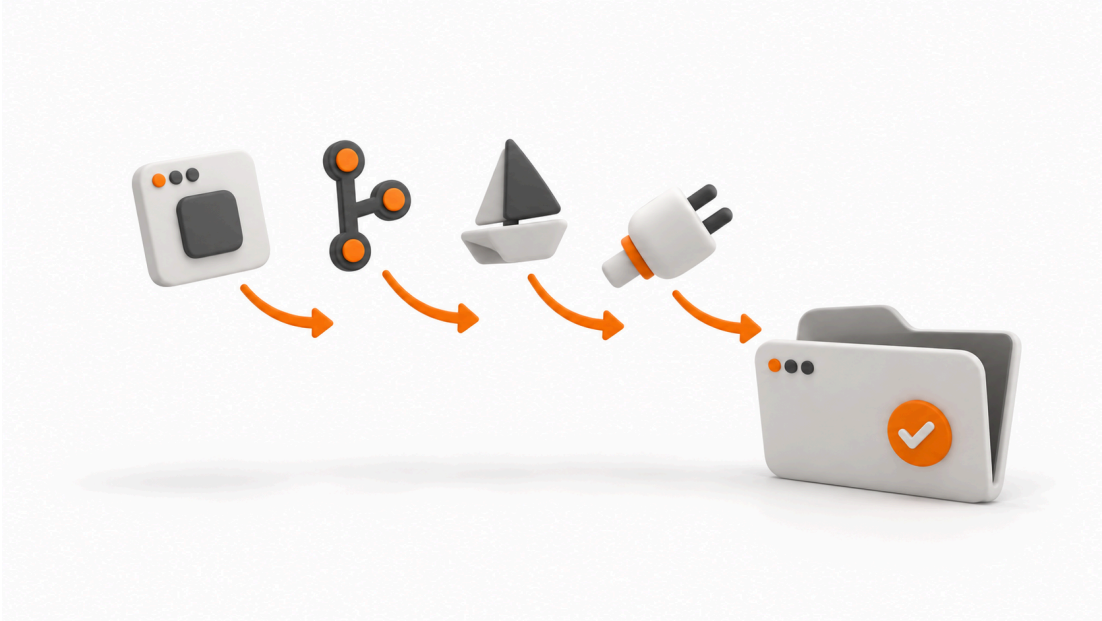
Starter check `npm install` `npm run preflight` `npm run dev`

FeedMe
Smart POS System

WORKSHOP SETUP

Open the starter before using project commands

Claude loads the workshop skills from the supplied project folder.



01 **Open the starter folder** in Claude Desktop.

02 **Install and check** the project from its terminal.

03 **Type /** and confirm both project skills appear.

```
npm install
```

```
npm run preflight
```

Skills available `/workshop-onboarding` `/wishful-programming`

Continue when preflight reports that all external tools and project skills are ready.

CHEAT SHEET

Workshop tool registry

The approved commands and connections available inside the supplied starter.

01 Slash commands

`/workshop-onboarding`	Check tools, accounts, and MCP
`/wishful-programming`	Wish to testable browser journey
`/mcp`	Check connection and OAuth status

02 MCP and connector

Vercel MCP	Projects and deployments
Build logs	Diagnose failed deployments
Runtime logs	Inspect deployed errors
Deployment rule	GitHub remains the source

03 Plugins and skills

Workshop skills	Bundled with the starter
Supabase skills	Optional after the core flow
Plugin menu	**+** then **Plugins**
Install policy	Mentor approval required

04 Command-line tools

`gh`	Authenticate, create, and push
`vercel`	Link project and connect Git
`npm`	Install, verify, and run locally
`git`	Track the team's source code

Type **/**, or open **+** and choose **Slash commands**, to see what is loaded in the current project.

Claude Desktop Vercel MCP Supabase skills

FeedMe
Smart POS System

CLAUDE CODE BASICS

Four tools participants will use

01

Skill

Reusable instructions for a specific workflow.

`/wishful-programming`

02

Slash command

A command or skill invoked at the start of a message.

`type / to browse`

03

Connector

An approved connection to an external service through MCP.

`Vercel MCP`

04

Plugin

An installable package that can add skills, agents, hooks, or MCP tools.

`+ > Plugins`

Claude Code Desktop documentation

IN CLAUDE DESKTOP

Using commands, connectors, and plugins

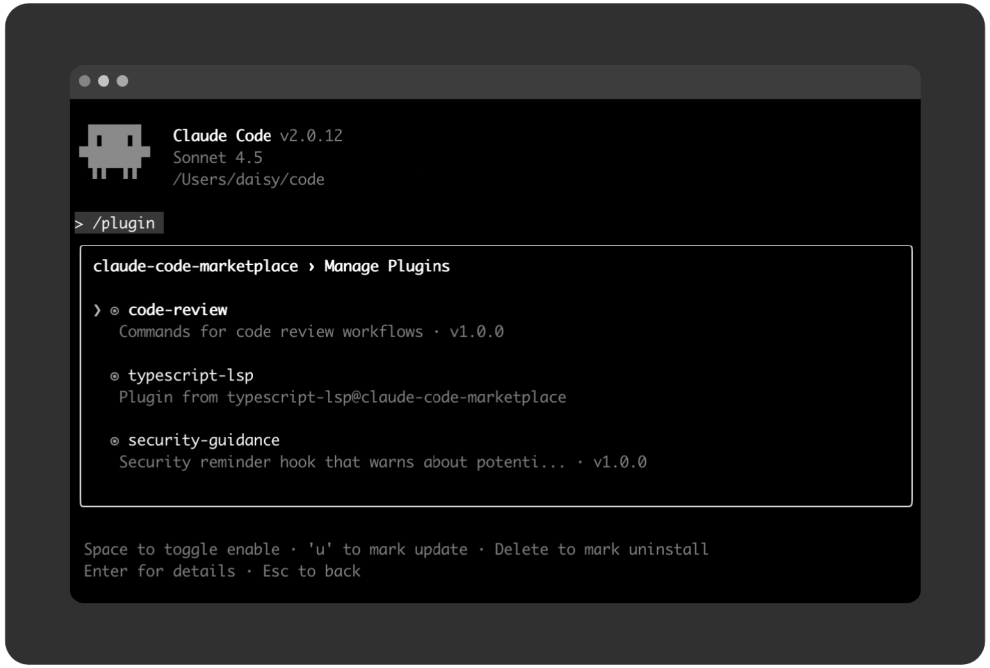
Browse Type `/`, or click **+** and choose **Slash commands**.

Run `/wishful-programming`, then answer one question at a time.

Connect Open the **+** menu and choose **Connectors**. Use `/mcp` to check Vercel.

Install Open the **+** menu and choose **Plugins**. Review the source and requested access first.

For this workshop, GitHub publishing still uses the authenticated `gh` CLI.



Official Anthropic example. Current Desktop manages plugins from the + menu. Source

FeedMe
Smart POS System

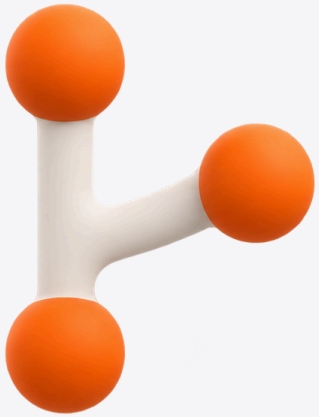
WORKSHOP OUTCOME

Three deliverables from the session



Working locally

A browser journey your tester can complete.



Saved on GitHub

A team-owned repository with clean history.



Live on Vercel

A public URL ready for the demo.

FeedMe
Smart POS System

GOAL SEEKING

Work backwards from the goal

Start with the finished outcome. Keep asking what must be true immediately before it.

01	**Goal**	What result do we want?
02	**Evidence**	What will prove the result happened?
03	**Conditions**	What must already be true?
04	**First step**	What can the team build or test now?

Example Goal: fewer missed orders. Evidence: staff see a confirmed order. Work backwards to the customer selecting, reviewing, and submitting an item.



FeedMe
Smart POS System

01

DESCRIBE

A focused project brief

`/wishful-programming`

Goal	The result the team wants
Evidence	What will prove it worked
First step	Where building or testing begins

Answer one question at a time. Claude will record the backwards chain.



02

BUILD

Plan approval and local verification

```
npm run verify
```

Plan	Claude proposes at most three tasks
Build	Use Next.js and local sample data
Verify	Run the checks before previewing

Your tester completes the happy path in a browser.



FeedMe
Smart POS System

03

PUBLISH

GitHub repository setup

```
gh auth status  git push origin main
```

Review	Check exactly what will be published
Approve	Create the repository and push
Confirm	Remote points to the team’s repo

Keep the repository private unless the team chooses public.



04

DEPLOY

Vercel Git deployment setup

```
vercel link  vercel git connect
```

Link	Connect the local project to Vercel
Deploy	Let the GitHub push trigger the build
Inspect	Use Vercel MCP for status and logs

Every later GitHub push creates another deployment.



PROMPT CARD

The four parts of a complete coding prompt

G **Goal** — The visible change you want.

C **Context** — Who needs it and where it appears.

R **Rules** — What Claude must keep or avoid.

D **Done when** — What the tester can visibly prove.

Example Add a confirmation screen after a student sends a request. Keep the current navigation. Use sample data. Done when the tester sees the support type, time, and reference number.

FeedMe
Smart POS System

WORKSHOP START

The first task is the project brief

Open the starter. Run onboarding first. Then use the Wishful Programming skill to define the first browser journey.

GitHub CLI Vercel MCP Vercel Git



FeedMe
Smart POS System